

UD 7. Webs dinámicas. Interacción cliente-servidor.

Índice de contenidos

1. Programación del cliente web.....	2
2. Comunicación asíncrona con el servidor web: AJAX.....	3
2.1. Uso del objeto XMLHttpRequest.....	3
Paso 1: Creación del XMLHttpRequest.....	3
Paso 2: Definir cómo se gestionará la respuesta.....	4
Paso 3: Enviar petición al servidor.....	5
2.2. Ventajas y desventajas.....	6
2.3. Ejemplos de interacción AJAX y PHP.....	7
2.3.1. Uso de procedimiento.....	7
2.3.2. Ejemplo de consulta a bases de datos.....	8
2.3.3. Ejemplo de AJAX con formularios.....	10
2.4. AJAX y XML.....	12
2.5. AJAX y JSON.....	14
2.6. Fetch API.....	17
2.7. CORS.....	19
2.7.1. Aceptar peticiones CORS en PHP.....	19
2.7.2. Configurar Apache para aceptar peticiones CORS.....	19

1. Programación del cliente web

Cuando comenzaste con el presente módulo, uno de los primeros conceptos que aprendiste es a diferenciar entre la **ejecución de código en el servidor web** y la **ejecución de código en el navegador o cliente web**.

Todo lo que has aprendido hasta el momento se ha centrado en la ejecución de código en el servidor web utilizando el lenguaje PHP. La otra parte importante de una aplicación web, la programación de código que se ejecute en el navegador, no forma parte de los contenidos de este módulo. Tiene su propio módulo dedicado, **Desarrollo Web en Entorno Cliente**.

Muchas de las aplicaciones web que existen en la actualidad tienen esos dos componentes: una parte de la aplicación, generalmente **la que contiene la lógica de negocio, se ejecuta en el servidor**; y otra parte de la aplicación, de menor peso, se ejecuta en el cliente. Existen incluso cierto tipo de aplicaciones web, como Google Docs, en las que gran parte de las funcionalidades que ofrecen se implementan utilizando programación del cliente web.

En la presente unidad vas a aprender cómo integrar estos dos componentes de una misma aplicación web: el código PHP que se ejecutará en el servidor, con el código que se enviará al cliente para que éste lo ejecute.



2. Comunicación asíncrona con el servidor web: AJAX

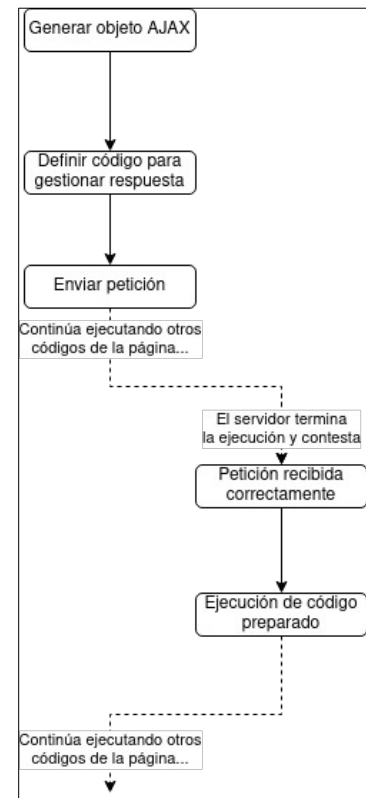
Una de las principales causas de la evolución de Javascript en los últimos tiempos es, sin duda, la tecnología **AJAX** ("Asynchronous Javascript and XML"). El término **AJAX** hace referencia a la posibilidad de una página web de establecer una comunicación con un servidor web y recibir una respuesta sin necesidad de que el navegador recargue la página.

El funcionamiento básicamente consiste en:

1. Crear un objeto AJAX.
2. Definir cómo se debe gestionar la respuesta cuando el servidor conteste.
3. Enviar la petición.
4. Continuar ejecutando código de la página
5. Cuando llegue la respuesta del servidor, gestionarla según lo establecido.

Entre las tareas que puedes llevar a cabo gracias a AJAX están:

- Actualizar el contenido de una página web sin necesidad de recargarla.
- Pedir y recibir información desde un servidor web manteniendo la página cargada en el navegador.
- Enviar información de la página a un servidor web en segundo plano.



2.1. Uso del objeto XMLHttpRequest

AJAX utiliza el objeto **XMLHttpRequest**, creado originariamente por Microsoft como parte de su librería MSXML, y que hoy en día se ha incorporado de forma nativa a todos los navegadores actuales. Pese al nombre del objeto, y a que la letra X de las siglas AJAX hace referencia a XML, la información que se transmite de forma asíncrona entre el navegador y el servidor web no es necesario que se encuentre en formato XML.

Paso 1: Creación del XMLHttpRequest

El primer paso consiste en la creación del objeto XMLHttpRequest. Como AJAX al fin y al cabo es Javascript se crea una variable objeto de la siguiente manera:

```
xmlhttp = new XMLHttpRequest();
```

Paso 2: Definir cómo se gestionará la respuesta

Antes de enviar la petición, hemos de indicar cómo debe gestionarse la respuesta (qué se debe hacer cuando el servidor conteste), y para ello utilizamos la propiedad **onreadystatechange**. Las peticiones AJAX van a pasar por distintos estados a lo largo de su corta vida. El estado de la petición se irá actualizando en la propiedad **readyState**, que admite los siguientes valores:

0	Petición no inicializada.
1	Conexión establecida con el servidor.
2	Petición recibida.
3	Procesando petición.
4	Petición finalizada y respuesta preparada.

Cada vez que se produce un cambio en el estado de la petición se ejecuta la función. Por tanto en una petición al servidor que se desarrolle satisfactoriamente se producirán al menos 4 cambios de estado y la función definida en la propiedad **onreadystatechange** deberá comprobar que estamos en el estado 4 (la petición ha terminado y la respuesta está lista) antes de proceder.

Además de comprobar el estado de la petición, será necesario confirmar si se ha producido una respuesta satisfactoria o si por el contrario el servidor ha encontrado problemas a partir del código HTTP de respuesta, que se almacena en la propiedad **status**.

Si todo ha ido bien, la propiedad **responseText** de **XMLHttpRequest** contiene la respuesta del servidor en forma de cadena de texto. En el siguiente ejemplo configuramos el objeto **XMLHttpRequest** para que, cuando el servidor conteste, incruste su respuesta en un elemento HTML con ID **myDiv**:

```
xmlhttp.onreadystatechange = function ()  
//Definimos función a ejecutar cada vez que cambie el estado del objeto y cuando llegue  
//respuesta...  
{  
    if (this.readyState == 4 && xmlhttp.status == 200) {  
        document.getElementById("myDiv").innerHTML = this.responseText;  
    }  
}
```

Si el servidor devuelve la información en formato XML, otra opción más acertada es utilizar la propiedad **responseXML** que devuelve la respuesta del servidor en forma de objeto XML. Para tratar esta respuesta podemos realizar un recorrido mediante la función **getElementsByTagName**.

```
xmlDoc = xmlhttp.responseXML; //Guardamos la respuesta en un objeto
txt = "";
//Recorremos el documento XML tratando cada nodo de tipo ARTISTA
x = xmlDoc.getElementsByTagName("ARTISTA");
for (i = 0; i < x.length; i++) {
    //Vamos acumulando en txt y finalmente lo colocamos en el div
    txt = txt + x[i].childNodes[0].nodeValue + "<br />";
}
document.getElementById("myDiv").innerHTML = txt;
```

Paso 3: Enviar petición al servidor

Para realizar esta acción utilizaremos los métodos **open()** y **send()** del objeto **XMLHttpRequest**.

open(method,url,async)

- **method**: Especifica el tipo de petición (GET o POST). POST tiene ciertas ventajas sobre GET como que evita que el navegador nos muestre páginas cacheadas. Si se utiliza GET, la información se envía a través de los parámetros de la URL. Si se utiliza POST, la información se envía a través del cuerpo.
- **url**: para indicar dónde podemos encontrar respuesta a nuestra petición. Esta url podrá ser un fichero estático (txt o xml), o un script PHP o ASP que construyan dinámicamente la respuesta.
- **async**: Indica si la petición se realizará de forma síncrona o asíncrona. Si realizamos la petición de forma síncrona (async=false *Deprecated) el Javascript permanecerá esperando hasta que el servidor envíe una respuesta. Esto tiene grandes consecuencias en términos de rendimiento y funcionalidad. Si en su lugar pedimos un comportamiento asíncrono, Javascript realizará la petición y no se quedará esperando por la respuesta. Mientras podrá seguir ejecutando otros script. Es necesario indicar qué tiene que realizar la página cuando llegue la petición del servidor. Para ello se utilizará el método onreadystatechange, donde podremos indicar el script a ejecutar cuando llegue respuesta.

send(string)

Envía la petición al servidor. El parámetro **string** se emplea para las peticiones POST. Para GET no es necesario.

abort()

Cancela una petición AJAX en curso.

Para enviar información con el método GET es necesario introducir los datos en la URL:

```
xmlhttp.open("GET", "pagina.php?nombre=Linus&apellido=Torvalds", true);
xmlhttp.send();
```

Para enviar la petición mediante el método **POST** de la misma forma en que se envía un formulario vamos a necesitar establecer un tipo de encabezado con el método **setRequestHeader** y además construir la misma cadena que construiríamos con el método GET pero en lugar de añadirla a la URL la pasaremos como parámetro en el método **send** del XMLHttpRequest.

```
xmlhttp.open("POST", "pagina.php", true);
xmlhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
xmlhttp.send("nombre=Linus&apellido=Torvalds");
```

En el siguiente ejemplo creamos una función Javascript que recibe dos números y hace una petición al script suma.php del servidor pasando esos números como parámetros. Cuando el servidor conteste, incluye dentro del div divResultado la respuesta que ha dado el servidor.

```
function sumaNumeros(num1, num2) {
    xmlhttp = new XMLHttpRequest();
    xmlhttp.onreadystatechange = function () {
        if (xmlhttp.readyState == 4 && xmlhttp.status == 200) { // Si la petición terminó y
            todo ha ido bien...
                document.getElementById("divResultado").innerHTML = xmlhttp.responseText;
            }
        }
    xmlhttp.open("GET", "suma.php?numero1=" + num1 + "&numero2=" + num2, true);
    xmlhttp.send();
}
```

2.2. Ventajas y desventajas

Ventajas:

- **Mejor experiencia de usuario.** Ajax permite que las páginas se modifiquen sin tener que volver a cargarse, dándole al usuario la sensación de que los cambios se producen instantáneamente. Este comportamiento es propio de los programas de escritorio a los que la mayoría de los usuarios están más acostumbrados. La experiencia se vuelve mucho más interactiva.
- **Optimización de recursos.** Al no recargarse la página se reduce el tiempo implicado en cada transacción. También se utiliza menos ancho de banda.
- **Alta compatibilidad.** Ajax es soportado por casi todas las plataformas Web.

Desventajas:

- Se produce un **incremento en la complejidad** de las páginas creadas con esta técnica, lo cual exige un mayor conocimiento de Javascript y los objetos de navegador creados para este fin.
- Las **páginas creadas de forma dinámica no son registradas en el objeto history** de navegador, lo cual impide que se pueda navegar (con las flechas de adelante y atrás) sobre los distintos estados que la página ha adquirido.
- Los cambios dinámicos obviamente no se registran en los motores de búsqueda que sólo pueden rastrear información estática.

- El uso de AJAX **aumenta el número de peticiones al servidor** (si bien estas son más ligeras).

2.3. Ejemplos de interacción AJAX y PHP

Con AJAX hacemos una petición al servidor y recibimos su respuesta. Su respuesta puede estar en varios formatos: XML, HTML, JSON...

A través de los siguientes ejemplos vamos a comprobar como funciona:

2.3.1. Uso de procedimiento

ejemplo1.html

Primero comprobamos si el campo input está vacío (`str.length == 0`). Si lo está, vaciamos `txtSugerencia` y terminamos.

Si el campo no está vacío se hace lo siguiente:

- Crear una petición XMLHttpRequest
- Crear la función que se va a ejecutar cuando tengamos la respuesta del servidor.
- Enviar la petición a un fichero PHP (`ejemplo1.php`) en el servidor.
- Fijate que le pasamos por GET un parámetro "q" con el contenido del input (la variable `str`).

```
<!-- ejemplo1.html -->
<html lang="es">
<head>
  <meta charset="utf-8">
  <script>
    function mostrarSugerencias(str) {
      if (str.length == 0) {
        document.getElementById("txtSugerencia").innerHTML = "";
        return;
      } else {
        var xmlhttp = new XMLHttpRequest();
        xmlhttp.onreadystatechange = function () {
          if (this.readyState == 4 && this.status == 200) {
            document.getElementById("txtSugerencia").innerHTML =
this.responseText;
          }
        };
        xmlhttp.open("GET", "ejemplo1.php?q=" + str, true);
        xmlhttp.send();
      }
    }
  </script>
</head>
<body>
  <p><strong>Empiece a escribir un nombre:</strong></p>
  <form>
    Nombre: <input type="text" onkeyup="mostrarSugerencias(this.value)">
  </form>
  <p>Sugerencias: <span id="txtSugerencia"></span></p>
</body>
</html>
```

ejemplo1.php

El script en php comprueba si lo que has escrito coincide con alguno de los nombres almacenados en el array e imprime una cadena con los nombres sugeridos.

```
<?php
// ejemplo1.php
// Array con nombres

$a = array(
    "Ana", "Begoña", "Conchita", "Diana", "Eva", "Flora", "Gunda", "Hilda", "Inga", "Johanna",
    "Karen", "Linda",
    "Nina", "Ofelia", "Petunia", "Amanda", "Raquel", "Cindy", "Dora", "Eva", "Evita", "Sandra",
    "Noelia", "Sara",
    "Violet", "Lisa", "Elisabeth", "Elena", "Laura", "Vicky"
);

// Almacenamos en $q el parámetro.
$q = $_REQUEST["q"];

$sugerencia = "";

// Comparamos $q con todos los nombres a ver si coincide con el principio ""
if ($q != "") {
    $longitudTexto = strlen($q);
    foreach ($a as $nombre) {
        if (strpos($nombre, substr($q, 0, $longitudTexto)) === 0) { // Comprobamos si coinciden.
            if ($sugerencia == "") {
                $sugerencia = $nombre;
            } else {
                $sugerencia .= ", $nombre";
            }
        }
    }
}

// Si no hay ningún resultado devolvemos "Sin sugerencias".
echo $sugerencia == "" ? "Sin sugerencias" : $sugerencia;
```

Actividad 7.1

Modifica el código anterior para que el script devuelva el ejemplo en una lista html (con `<li>`) en lugar de separarlo por comas.

2.3.2. Ejemplo de consulta a bases de datos

Ajax nos permite poder hacer consultas a la base de datos a través de PHP. En este ejemplo vamos a modificar de forma dinámica elementos de la página consultando información en la base de datos.

En concreto vamos a crear un formulario en el que tenemos un campo select para seleccionar una comunidad autónoma y otro select desactivado para seleccionar la provincia.

Vamos a hacer uso de una base de datos en la que tenemos la información de las comunidades autónomas de España, de sus provincias y de sus municipios. Podrás encontrar sus archivos en Moodle.

¡Atención!

Carga en tu base de datos los archivos comunidades.sql, provincias.sql y municipios.sql para que te creen las tablas correspondientes. Debes hacerlo en ese orden.

ejemplo2formulario.php

- En el siguiente código tenemos una función que comprueba el valor del **select** cuando este cambia. Si está vacío, borra el HTML dentro del **select** de provincias y lo desactiva.
- Si hay alguna opción seleccionada, hace la petición al servidor pasándole por **GET** el id de la comunidad seleccionada y rellena selProvincia con la respuesta del servidor, además de activarlo.
- Fíjate que este código soporta versiones antiguas de Internet Explorer.

```
<?php
// ejemplo2formulario.php
$dbd = new PDO('mysql:host=localhost;dbname=dwes;charset=utf8', 'dwes', 'abc123');
?>
<html lang="es">

<head>
  <meta charset="utf-8">
  <script>
    function actualizaProvincias(idComunidad) {
      if (idComunidad == "") {
        document.getElementById("selProvincia").innerHTML = "";
        document.getElementById("selProvincia").disabled = true;
        return;
      } else {
        // Este ejemplo soporta navegadores antiguos
        if (window.XMLHttpRequest) {
          // código para IE7+, Firefox, Chrome, Opera, Safari
          xmlhttp = new XMLHttpRequest();
        } else {
          // código para IE6, IE5
          xmlhttp = new ActiveXObject("Microsoft.XMLHTTP");
        }
        xmlhttp.onreadystatechange = function () {
          if (this.readyState == 4 && this.status == 200) {
            document.getElementById("selProvincia").innerHTML = this.responseText;
            document.getElementById("selProvincia").disabled = false;
          }
        };
        xmlhttp.open("GET", "ejemplo2servidor.php?id=" + idComunidad, true);
        xmlhttp.send();
      }
    }
  </script>
</head>

<body>
  <form>
    <label>Comunidad</label>
    <select name="selComunidad" onchange="actualizaProvincias(this.value)">
      <option value="">Selecione...</option> <!-- Opción vacía -->
      <?php
        $resultado = $dbd->query('SELECT id, comunidad FROM comunidades ORDER BY comunidad ASC');
        while ($comunidad = $resultado->fetch()) {
          echo ' <option value="' . $comunidad['id'] . '">' . $comunidad['comunidad'] .
'</option>';
        }
      <?>
    </select>
    <label>Provincia</label>
    <select id="selProvincia" disabled>
    </select>
  </form>
</body>
</html>
```

ejemplo2servidor.php

El siguiente código se conecta a la base de datos, comprueba el id que le mandan e imprime el `<option>` para cada provincia perteneciente a la comunidad seleccionada.

```
<?php
//ejemplo2servidor.php
$bd = new PDO('mysql:host=localhost;dbname=dwes;charset=utf8', 'dwes', 'abc123');

if (isset($_GET['id'])) {
    echo '<option value="">Selecione...</option>'; // Imprimimos opción vacía

    $consulta = $bd->prepare("SELECT id, provincia FROM provincias WHERE comunidad_id = :id");
    $consulta->execute(['id' => $_GET['id']]);
    while ($provincia = $consulta->fetch()) {
        echo '<option value="' . $provincia['id'] . '">' . $provincia['provincia'] .
    '</option>';
    }
} else {
    echo "";
}
```

Actividad 7.2

Añade un select para Municipios y haz que funcione de forma similar a comunidades y provincias.

2.3.3. Ejemplo de AJAX con formularios

Una de las principales aplicaciones de AJAX consiste en **validar formularios** antes de su envío.

Como sabemos, debemos realizar la validación de los formularios en el servidor, para controlar que no se procesen solicitudes no válidas. Sin embargo, realizar validación también en el cliente es muy recomendable, ya que así se puede avisar al usuario de qué errores tiene antes de que haga un envío que va a ser rechazado.

Aspectos como que la longitud de un campo tenga un mínimo, que un campo no esté vacío, que la contraseña introducida cumpla unos requisitos... se pueden controlar en el cliente pero hay otros para los cuales necesitamos del servidor y, si queremos controlarlos también en el cliente, deberemos hacer uso de AJAX.

En el siguiente ejemplo contamos con una base de datos con información de los usuarios de una web (id, username, nombre y email) y tenemos un formulario con un campo email. A través de AJAX enviamos el contenido del campo a un script que nos avisa si el correo está ya registrado o no.

¡Atención!

Carga en tu base de datos el archivo users.sql para crear la tabla correspondiente.

ejemplo3.html

```
<!-- ejemplo3.html -->
<html lang="es">

<head>
  <meta charset="utf-8">
  <script>
    function comprobar(str) {
      if (str.length == 0) {
        document.getElementById("error").innerHTML = "";
        return;
      } else {
        var xmlhttp = new XMLHttpRequest();
        xmlhttp.onreadystatechange = function () {
          if (this.readyState == 4 && this.status == 200) {
            if (this.responseText != "OK") {
              document.getElementById("error").innerHTML = this.responseText;
            } else {
              document.getElementById("error").innerHTML = "";
            }
          }
        };
        xmlhttp.open("GET", "ejemplo3.php?q=" + str, true);
        xmlhttp.send();
      }
    }
  </script>
  <style>
    #error {
      color: red;
      font-weight: bold;
    }
  </style>
</head>

<body>
  <p><strong>Escriba un correo electrónico:</strong></p>
  <form>
    Correo: <input type="text" onkeyup="comprobar(this.value)"> <span id="error"></span>
  </form>
</body>

</html>
```

ejemplo3.php

```
<?php
//ejemplo3.php
$dbd = new PDO('mysql:host=localhost;dbname=dwes;charset=utf8', 'dwes', 'abc123');

if (isset($_GET['q'])) {
  $consulta = $dbd->prepare("SELECT id FROM users WHERE lower(email) = lower(:email)");
  $consulta->execute(['email' => $_GET['q']]);
  if ($usuario = $consulta->fetch()) {
    echo "El correo ya existe";
  } else {
    echo "OK";
  }
} else {
  echo "";
}
```

Actividad 7.3

Añade un campo "Nombre de usuario" y comprueba que el nombre de usuario introducido no exista.

2.4. AJAX y XML

Hasta ahora hemos estado utilizando AJAX y ya sabemos que la X viene de XML, pero hasta ahora XML no lo hemos utilizado.

Estamos utilizando las respuestas del servidor como texto plano. Esto es muy cómodo si la respuesta viene preparada para ser incrustada dentro de una etiqueta HTML o queremos hacer debug, pero habrá ocasiones en la que necesitemos intercambiar la información en un formato que nos permita navegar por ella, como XML.

La tecnología AJAX se desarrolló pensando en XML, por lo que tenemos una propiedad similar a `this.responseText`, **`this.responseXML`**, que nos permite tratar la respuesta como un archivo XML. Gracias a esto podemos usar funciones de Javascript para acceder directamente a la información que necesitamos.

En el siguiente ejemplo tenemos un formulario para seleccionar personajes de la serie Padre de Familia. Cuando seleccionamos un personaje, se obtiene una respuesta XML con su información, la cual se utiliza para rellenar elementos HTML:

¡Atención!

Carga en tu base de datos el archivo familyguy.sql para crear la tabla correspondiente.
ejemplo4formulario.php

```
$bd = new PDO('mysql:host=localhost;dbname=dwes;charset=utf8', 'dwes', 'abc123');
?>
<html>

<head>
    <meta charset="utf8" />
    <script src="ejemplo4.js"></script>
</head>

<body>
    <form>
        <label for="selPersonaje">Seleccione un usuario:</label>
        <select name="selPersonaje" onchange="mostrarPersonaje(this.value)">
            <option value="">Seleccione...</option>
            <?php
                $resultado = $bd->query('SELECT id, nombre FROM familyguy ORDER BY nombre ASC');
                while ($personaje = $resultado->fetch()) {
                    echo '<option value="' . $personaje['id'] . '>' . $personaje['nombre'] .
'</option>';
                }
            ?>
        </select>
    </form>
    <div id="ficha" style="display: none;">
        <h2><span id="spnNombre"></span> <span id="spnApellido"></span></h2>
        <p><strong>Profesión</strong>: <span id="spnTrabajo"></span></p>
        <p><strong>Edad</strong>: <span id="spnEdad"></span></p>
        <p><strong>Ciudad</strong>: <span id="spnCiudad"></span></p>
    </div>
</div>

</body>
</html>
```

ejemplo4.js

Este código almacena la respuesta en formato XML en la variable `xmlDoc` y utiliza algunos métodos propios de los objetos XML. Con `getElementsByTagName(nombre)` obtenemos los elementos de la etiqueta indicada, con `childNodes[0]` indicamos que queremos el primer elemento (en este caso sólo hay uno) y con `nodeValue` indicamos que queremos el valor de ese elemento:

```
// ejemplo4.js
function mostrarPersonaje(str) {
    if (str.length == 0) {
        document.getElementById("spnNombre").innerHTML = "";
        document.getElementById("spnApellido").innerHTML = "";
        document.getElementById("spnTrabajo").innerHTML = "";
        document.getElementById("spnEdad").innerHTML = "";
        document.getElementById("spnCiudad").innerHTML = "";
        document.getElementById("ficha").style.display = "none";
        return;
    } else {
        var xmlhttp = new XMLHttpRequest();
        xmlhttp.onreadystatechange = function () {
            if (this.readyState == 4 && this.status == 200) {
                var xmlDoc = this.responseXML;
                document.getElementById("spnNombre").innerHTML =
                    xmlDoc.getElementsByTagName("nombre")[0].childNodes[0].nodeValue;
                document.getElementById("spnApellido").innerHTML =
                    xmlDoc.getElementsByTagName("apellido")[0].childNodes[0].nodeValue;
                document.getElementById("spnTrabajo").innerHTML =
                    xmlDoc.getElementsByTagName("trabajo")[0].childNodes[0].nodeValue;
                document.getElementById("spnEdad").innerHTML =
                    xmlDoc.getElementsByTagName("edad")[0].childNodes[0].nodeValue;
                document.getElementById("spnCiudad").innerHTML =
                    xmlDoc.getElementsByTagName("ciudad")[0].childNodes[0].nodeValue;
                document.getElementById("ficha").style.display = "block";
            }
        };
        xmlhttp.open("GET", "ejemplo4servidor.php?q=" + str, true);
        xmlhttp.send();
    }
}
```

ejemplo4servidor.php

El resultado de este script no es HTML ni texto, sino XML. Por ello añadimos unas líneas especiales de cabecera que indican que el contenido es XML, para que el navegador no use la caché, se indica fecha de caducidad antigua e instrucciones específicas de que no debe ser cacheado:

```
<?php
// ejemplo4servidor.php
header('Content-Type: text/xml'); // Esta línea indica que la respuesta es XML
header("Cache-Control: no-cache, must-revalidate"); // Esta línea ayuda a que la respuesta no se incluya en caché
// Fecha caducada
header("Expires: Mon, 26 Jul 1997 05:00:00 GMT"); // Esta línea ayuda a que la respuesta no se incluya en caché

$q = $_GET["q"];
$bd = new PDO('mysql:host=localhost;dbname=dwes;charset=utf8', 'dwes', 'abc123');
echo "<?xml version='1.0' encoding='ISO-8859-1'?><persona>";
$consulta = $bd->prepare("SELECT nombre, apellido, edad, ciudad, trabajo FROM familyguy WHERE id = :id");
$consulta->execute(['id' => $_GET['q']]);
if ($persona = $consulta->fetch()) {
    echo "<nombre>" . $persona['nombre'] . "</nombre>";
    echo "<apellido>" . $persona['apellido'] . "</apellido>";
    echo "<edad>" . $persona['edad'] . "</edad>";
    echo "<ciudad>" . $persona['ciudad'] . "</ciudad>";
    echo "<trabajo>" . $persona['trabajo'] . "</trabajo>";
}
echo "</persona>";
```

Actividad 7.4

Prueba a acceder a `ejemplo4servidor.php?q=1` para comprobar cómo devuelve la respuesta el servidor.

Modifica el `.html` y el `.js` para utilizar solamente el nombre, el apellido y el trabajo, sin cambiar nada en el script `php`.

Actividad 7.5

Haz una página que muestre un select con los productos de la base de datos de la UT 3. Una vez se seleccione un producto, obtén con AJAX y XML su nombre, descripción y precio y muéstralo en una ficha.

2.5. AJAX y JSON

JSON (Javascript Object Notation) es una forma sencilla y ligera de transformar a cadena de texto un objeto Javascript (y al contrario, obtener un objeto a partir de su representación en cadena de texto). Actualmente está convirtiéndose en el formato de intercambio de información más popular frente a XML y es soportado por otros lenguajes de programación.

Presenta las siguientes características:

- Los datos se componen de nombre (entre comillas dobles) / valor. Ejemplos son:
 - **números**: 5
 - **strings** (comillas dobles): "patata"
 - **booleanos**: (true/false)
 - **arrays** ([...]): ["juan", "pedro", "jacinto"]
 - **objetos** ({...}): {"departamento":8, "nombredepto":"Ventas"}
 - **null**
- Cada dato va separado por una coma.
- Los objetos pueden contener muchos pares de nombre y valores.
- Los arrays pueden contener muchos objetos.
- Los ficheros tienen extensión `.json`.
- El tipo MIME asociado es `application/json`.

En Javascript, para transformar una variable en JSON se usa la función:

```
cadena = JSON.stringify(variable)
```

Y para recuperar una variable a partir de JSON se usa:

```
variable = JSON.parse(cadena)
```

De forma similar, en PHP tenemos `json_encode()` y `json_decode()`:

```
$cadena = json_encode($variable);  
$variable = json_decode($cadena);
```

El siguiente ejemplo es una adaptación del ejemplo anterior, usando JSON en lugar de XML:

ejemplo5formulario.php

```
<?php  
// ejemplo5formulario.php  
$bd = new PDO('mysql:host=localhost;dbname=dwes;charset=utf8', 'dwes', 'abc123');  
?>  
<html>  
  
<head>  
    <meta charset="utf8" />  
    <script src="ejemplo5.js"></script>  
</head>  
  
<body>  
    <form>  
        <label for="selPersonaje">Seleccione un usuario:</label>  
        <select name="selPersonaje" onchange="mostrarPersonaje(this.value)">  
            <option value="">Seleccione...</option>  
            <?php  
                $resultado = $bd->query('SELECT id, nombre FROM familyguy ORDER BY nombre ASC');  
                while ($personaje = $resultado->fetch()) {  
                    echo '<option value="' . $personaje['id'] . '">' . $personaje['nombre'] .  
'</option>';  
                }  
            ?>  
        </select>  
    </form>  
    <div id="ficha" style="display: none;">  
        <h2><span id="spnNombre"></span> <span id="spnApellido"></span></h2>  
        <p><strong>Profesión</strong>: <span id="spnTrabajo"></span></p>  
        <p><strong>Edad</strong>: <span id="spnEdad"></span></p>  
        <p><strong>Ciudad</strong>: <span id="spnCiudad"></span></p>  
    </div>  
</div>  
</body>  
</html>
```

ejemplo5.js

```
// ejemplo5.js
function mostrarPersonaje(str) {
    if (str.length == 0) {
        document.getElementById("spnNombre").innerHTML = "";
        document.getElementById("spnApellido").innerHTML = "";
        document.getElementById("spnTrabajo").innerHTML = "";
        document.getElementById("spnEdad").innerHTML = "";
        document.getElementById("spnCiudad").innerHTML = "";
        document.getElementById("ficha").style.display = "none";
        return;
    } else {
        var xmlhttp = new XMLHttpRequest();
        xmlhttp.onreadystatechange = function () {
            if (this.readyState == 4 && this.status == 200) {
                persona = JSON.parse(this.responseText);
                document.getElementById("spnNombre").innerHTML = persona.nombre
                document.getElementById("spnApellido").innerHTML = persona.apellido
                document.getElementById("spnTrabajo").innerHTML = persona['trabajo'] //
                Podemos usar también este formato
                document.getElementById("spnEdad").innerHTML = "Edad: " + persona['edad'] //
                Podemos usar también este formato
                document.getElementById("spnCiudad").innerHTML = "<br/>De: " + persona.ciudad
                document.getElementById("ficha").style.display = "block";
            }
        };
        xmlhttp.open("GET", "ejemplo5servidor.php?q=" + str, true);
        xmlhttp.send();
    }
}
```

ejemplo5servidor.php

```
<?php
// ejemplo5servidor.php
header('Content-Type: application/json'); // Esta línea indica que la respuesta es JSON
header("Cache-Control: no-cache, must-revalidate"); // Esta línea ayuda a que la respuesta no
se incluya en caché
// Fecha caducada
header("Expires: Mon, 26 Jul 1997 05:00:00 GMT"); // Esta línea ayuda a que la respuesta no se
incluya en caché

$q = $_GET["q"];
$db = new PDO('mysql:host=localhost;dbname=dwes;charset=utf8', 'dwes', 'abc123');

$consulta = $db->prepare("SELECT nombre, apellido, edad, ciudad, trabajo FROM familyguy WHERE
id = :id");
$consulta->execute(['id' => $_GET['q']]);
if ($persona = $consulta->fetch()) {
    echo json_encode($persona);
}
```

Actividad 7.6

Accede a [ejemplo5.phpservidor?q=1](#) para comprobar cómo devuelve la respuesta el servidor.

Actividad 7.7

Modifica el ejercicio anterior de la base de datos de productos para utilizar JSON en lugar de XML.

2.6. Fetch API

El API Fetch es un nuevo estándar que viene a dar una alternativa para interactuar por HTTP, con un diseño moderno, basado en promesas, con mayor flexibilidad y capacidad de control a la hora de realizar llamadas al servidor. Se prevee que sustituya a XMLHttpRequest.

Su soporte se está implementando de forma progresiva en los navegadores web. Para aquellos navegadores que no lo soporten, podemos utilizar el [polyfill publicado por Github](#). Una de las opciones que **no soporta** es la de **cancelar peticiones**.

Para saber más ...

Un **polyfill** es una librería que proporciona a un navegador antiguo compatibilidad con funcionalidades que versiones más novedosas si soportan.

Una de las características más importantes del API Fetch es que utiliza **promesas**, es decir, devuelve un objeto con dos métodos, uno **then()** y otro **catch()**, a la que pasaremos una función que será invocada cuando se obtenga la respuesta o se produzca un error.

Otro aspecto importante que hay que comprender es que para obtener el body o cuerpo del mensaje devuelto por el servidor deberemos obtener una segunda promesa por medio de los métodos del objeto Response (obtener la información del texto, del XML, del json... de la respuesta). Por ello será muy **habitual ver dos promesas encadenadas**, una para el fetch() y otra con el retorno del método que utilicemos para obtener el body.

Para este ejemplo vamos a utilizar las opciones que nos ofrece httpbin.org para comprobar el funcionamiento de nuestros clientes HTTP.

```
fetch('https://httpbin.org/ip')
  .then(function(response) {
    return response.text(); //Obtenemos el texto
  })
  .then(function(data) { // data es el texto que ha retornado el .then anterior.
    console.log('data = ', data);
  })
  .catch(function(err) {
    console.error(err);
  });
```

¿Qué hemos hecho?:

1. Hemos llamado a fetch() con la URL a la que queremos acceder como parámetro
2. Esta llamada nos devuelve una promesa
3. El método then() de esa promesa nos entrega un objeto response

4. Del objeto response llamamos al método text() para obtener el cuerpo retornado en forma de texto
5. Nos devuelve otra promesa que se resolverá cuando se haya obtenido el contenido
6. El método then() de esa promesa recibe el cuerpo devuelto por el servidor en formato de texto
7. Hemos incluido un catch() por si se produce algún error

Si modificáramos el ejemplo 5 para utilizar Fetch API en lugar de XMLHttpRequest, quedaría de la siguiente manera:

ejemplo5fetch.js

```
// ejemplo5fetch.js
function mostrarPersonaje(str) {
  if (str.length == 0) {
    document.getElementById("spnNombre").innerHTML = "";
    document.getElementById("spnApellido").innerHTML = "";
    document.getElementById("spnTrabajo").innerHTML = "";
    document.getElementById("spnEdad").innerHTML = "";
    document.getElementById("spnCiudad").innerHTML = "";
    document.getElementById("ficha").style.display = "none";
    return;
  } else {
    fetch('ejemplo5servidor.php?q=' + str).then(function (response) {
      // Convertimos a JSON
      return response.json(); // Este response.json() que devolvemos...
    }).then(function (persona) { // ...lo recibimos en esta función en el parámetro
      "persona"
      document.getElementById("spnNombre").innerHTML = persona.nombre
      document.getElementById("spnApellido").innerHTML = persona.apellido
      document.getElementById("spnTrabajo").innerHTML = persona['trabajo'] // Podemos
      usar este formato
      document.getElementById("spnEdad").innerHTML = persona['edad'] // Podemos usar
      también este formato
      document.getElementById("spnCiudad").innerHTML = persona.ciudad
      document.getElementById("ficha").style.display = "block";
    });
  }
}
```

Para saber más ...

En esta [comparación de Google Trends](#) puedes comprobar cómo ha evolucionado la búsqueda en Google de los términos xmlhttprequest y fetch api

Actividad 7.8

Modifica el ejercicio anterior de la base de datos de productos con JSON para usar la Fetch API.

2.7. CORS

Es posible que al realizar los ejercicios anteriores te hayas encontrado con un mensaje de error que menciona el acrónimo CORS. CORS significa **Cross-origin resource sharing** (intercambio de recursos de origen cruzado) y se refiere a la situación en que un dominio está solicitando recursos restringidos a otro.

Una página web puede incrustar imágenes, hojas de estilo, scripts Javascript... de otra página web sin ningún tipo de problema, pero hay otros recursos (tipografías, peticiones AJAX...) que por defecto están restringidos para evitar un uso malicioso.

Para evitar este problema debemos configurar nuestra página servidora para que acepte peticiones de otros dominios, ya sea de uno en concreto o de cualquiera. Para ello tenemos dos opciones:

2.7.1. Aceptar peticiones CORS en PHP

Podemos añadir líneas en nuestro script PHP para que acepte las peticiones. Simplemente debemos añadir las siguientes tres líneas al inicio:

```
header('Access-Control-Allow-Origin: *');  
header("Access-Control-Allow-Headers: Origin, X-Requested-With, Content-Type, Accept");  
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE');
```

Esta configuración acepta cualquier petición debido al *. Si quisiéramos aceptar únicamente peticiones AJAX del dominio `juntadeandalucia.es` deberíamos especificar:

```
header('Access-Control-Allow-Origin: https://www.juntadeandalucia.es');
```

2.7.2. Configurar Apache para aceptar peticiones CORS

Podemos configurar nuestro servidor Apache para que cualquier script que tengamos acepte las peticiones. Para ello añadimos la siguiente línea a nuestro archivo de configuración del host en `/etc/apache2/sites-available`:

```
Header set Access-Control-Allow-Origin ""
```

El archivo podría quedar así:

La línea listen obliga a Apache a estar atento al puerto indicado

```
Listen 8888  
  
<VirtualHost *:8888>  
    ServerAdmin admin@localhost  
    DocumentRoot /var/www/dwes  
  
    ErrorLog ${APACHE_LOG_DIR}/error.log  
    CustomLog ${APACHE_LOG_DIR}/access.log combined  
  
    Header set Access-Control-Allow-Origin ""  
</VirtualHost>
```

Por último quedaría asegurarse de que el módulo Headers de Apache está cargado y reiniciar el servicio para que cargue los cambios:

```
sudo a2enmod headers  
sudo systemctl restart apache2
```